KIWi

KI-Wissensassistenz

AI-Powered Knowledge Assistance for the THWS CampusPortal

1. Project overview and design principles

KIWi (KI-Wissensassistenz) is envisioned as an AI-powered knowledge assistant for the THWS CampusPortal. The proposed approach uses Retrieval-Augmented Generation (RAG): the application retrieves relevant passages from approved university documentation and supplies those passages to a language model to produce a grounded answer.

The design assumes a local-first deployment, approved knowledge sources, and answers that show their sources. The architecture below is a practical starting point, not a claim that every component or requirement has already been approved.

2. End-to-end RAG architecture

The system consists of two connected pipelines: an offline ingestion/indexing pipeline and an online query pipeline. The ingestion pipeline prepares the knowledge base; the query pipeline runs whenever a user submits a question.

2.1 Architecture diagram

1. Approved knowledge sources

THWS/HISinOne guides, FAQs, regulations and process documentation

↓

2. Document processing

Load → extract text → clean → split into chunks → attach metadata

↓

3. Embedding model

Convert each text chunk into a vector representing semantic meaning

↓

4. Vector database

Store embeddings, chunk text, document IDs, page/section references and metadata

↓

5. User question

Question submitted through the assistant interface

↓

6. Retrieval

Embed question and retrieve relevant chunks; optionally combine keyword search and reranking

↓

7. Context construction

Select useful passages and preserve source references within the model context limit

↓

8. Local LLM

Generate an answer using retrieved evidence and instructions not to invent missing facts

↓

9. Validation

Check grounding and citations; abstain or ask for clarification if evidence is insufficient

↓

10. Response

Display a clear answer with links or references to source documents and pages/sections

2.2 Offline indexing pipeline

1. Inventory approved source documents and record ownership, version, update frequency and access restrictions.
2. Parse PDF, Word and HTML sources; preserve useful structure such as headings, tables and page numbers.
3. Clean boilerplate and split text into semantically coherent chunks.
4. Attach metadata such as document title, source URI, version, page, section and permissions.
5. Generate embeddings and store the vectors and original chunk text in the vector database.
6. Re-index new or changed files and remove obsolete versions according to a controlled update process.

2.3 Online question-answering pipeline

7. Accept and validate the user's question.
8. Apply authentication, authorization and any document-access filters required by the deployment.
9. Generate a question embedding and retrieve the most relevant chunks.
10. Optionally combine vector retrieval with keyword search (BM25) and rerank candidate passages.
11. Check whether the evidence is sufficiently relevant and complete.
12. Construct a prompt containing the question, retrieved passages and explicit grounding instructions.
13. Run inference using the local LLM.
14. Validate citations and answer support; if necessary, abstain, clarify or explain that the sources do not contain the answer.
15. Return the answer with source references and collect privacy-conscious quality metrics.

3. Workflow diagram

The diagram below illustrates the data flow, including the two main decision points: whether retrieval provides enough evidence and whether the generated answer is sufficiently supported.

Approved THWS documents

Begin with sources approved for this use case.

↓

Load → extract → clean

Parse files and preserve relevant structure.

↓

Chunk → attach metadata

Create passages with stable source references.

↓

Generate embeddings

Encode each passage as a vector.

↓

Persist in vector database

Store vectors, text and metadata.

↓

User submits a question

Start the online query pipeline.

↓

Validate access → embed question

Apply relevant access filters and create query vector.

↓

Retrieve → optionally rerank

Search for relevant evidence.

↓

Enough evidence?

If no, ask for clarification or abstain; if yes, continue.

↓

Build prompt → local LLM

Generate a response grounded in retrieved passages.

↓

Validate answer and citations

Check that references exist and support the answer.

↓

Supported?

If no, abstain or revise; if yes, return the response.

↓

Answer with source links

Show the answer and its evidence to the user.

4. Recommended technology stack

Component	Recommended choice	Reason for choice
Programming language	Python 3.11	Mature AI ecosystem, strong document-processing support and straightforward integration.
Document parsing	PyMuPDF, python-docx, BeautifulSoup	Extract text from PDF, Word and HTML. Add OCR only if scanned documents are in scope.
RAG orchestration	LlamaIndex or a custom Python pipeline	LlamaIndex speeds up ingestion and retrieval prototyping; a custom pipeline offers tighter control and fewer abstractions.
Embedding model	A multilingual E5-family model (candidate)	Creates vectors for passages and queries; multilingual support may help with German and English content. Validate on representative questions.
Vector database	ChromaDB	Simple local persistence, metadata filtering and a low-friction development setup.
Local LLM runtime	Ollama	Convenient local model management and a straightforward interface for testing models.
Generation model	Evaluate suitable Qwen-, Mistral- or Llama-family instruct models	Candidate model families for instruction following and question answering. Benchmark exact model versions on available hardware.
Backend API	FastAPI	Typed request/response schemas, validation and straightforward endpoints for questions and health checks.
User interface		Quick to implement a chat interface with source citations and feedback controls.
Optional retrieval enhancement	BM25 + vector search	Keyword search helps with exact terms and module names; vector search helps with semantic matches. Reranking can improve top-result relevance.
Evaluation	RAGAS plus custom tests and human review	Provides evaluation measures for retrieval and generation; source-specific checks and human assessment remain necessary.
Packaging	Docker Compose	Reproducible multi-service development and deployment, subject to compatibility with the target host.
Version control and tests	GitHub, pytest and CI checks	Supports team collaboration, repeatable tests and regression detection.